BUILDING AI INFRASTRUCTURE · PART 7 OF 7

Part 7: One Platform Across All Four

Chapter summary for beginners

In one sentence: turn four separate platforms (AWS, Azure, Google Cloud and on-prem) into one, with one way to deploy, one place to look, one set of rules, one view of cost, and a recovery plan you actually test.

Video length: about 9 minutes · **Code: this repo** · **Cost:** whatever your clusters cost; the fleet layer itself is free, open-source software

The five things that make it one platform

#	What	One-line meaning	Where in this repo
1	**GitOps**	Every cluster pulls its desired state from Git	`flux/clusters/*`, `argocd/clusters/*`
2	**One observability**	OpenTelemetry everywhere, one place to look	`flux/infrastructure/telemetry`, `observability/`
3	**One identity and policy**	The same rules, enforced in every cluster	`flux/infrastructure/policy`
4	**One cost view**	Spend by team and model, across all four	OpenCost, `observability/prometheus/gpu-cost-rules.yaml`
5	**A recovery plan**	What happens when a region or provider fails	`docs/disaster-recovery.md`

1 · GitOps: Git is the source of truth

1. **Change in Git:** a pull request.
2. **Review and checks:** tests, policy and security scans.
3. **Merge:** approved and recorded (who changed what, and why).
4. **Clusters pull:** Flux or Argo CD applies the change.
5. **Drift fixed:** manual edits get reverted automatically.

Nobody runs `kubectl` against production by hand. The fleet has one folder per cluster: `dev-eks`, `staging-gke`, `prod-aks` and `onprem-openshift`. **Adding a cluster is adding a folder. Promoting a change is a pull request.**

2 · One observability pipeline

There's an OpenTelemetry **agent on every node** and a **gateway per cluster**. They send traces to Tempo, logs to Loki and metrics to Prometheus, and you view it all in Grafana. Teams instrument their apps once, and it

works everywhere. Every cluster has the same SLOs, the same burn-rate alerts and the same runbooks, so whoever is on call knows what to do wherever the problem is.

3 · One set of rules, everywhere

Policy as code (Kyverno)	Identity	Supply chain
No containers running as root	Single sign-on for people	Scan every image
No images tagged `latest`	Workload identity for pods	Sign images, keep an SBOM
CPU and memory limits required	No long-lived keys anywhere	Only signed images can run

In this repo, policies start in **Audit** mode: they report problems but block nothing. Switch each one to **Enforce** once its report is clean.

4 · One view of cost

- **Per team:** label every workload with `team` and `cost-center`, so every dollar has an owner. A policy here requires those labels.
- **Per model:** track cost per 1,000 predictions, not just per server. That's the number the business understands.
- **Idle GPUs:** these are the biggest waste. This repo alerts with `GPUIdle`.

The tools are OpenCost in every cluster, plus each cloud's billing export, combined into one report. On-prem, spread the hardware cost over its lifetime.

5 · When something big fails

Active-passive (start here)	Active-active
One primary, one standby elsewhere	Two sites serve traffic at once
Data copied on a schedule	Global load balancing + data sync
Simpler and cheaper	Hardest and most expensive

Write your **RTO** (how long you can be down) and **RPO** (how much data you can lose), and then **run the drill**.

> A recovery plan only counts after you have tested it. Until then, it's a hope, not a plan.

The whole series in one picture

Part 1 covered the 8 blocks. Part 2 ran all 8 on a laptop. Parts 3–5 moved the platform to AWS, Azure and Google Cloud, and Part 6 to on-prem. Part 7 made it one platform.

> **Design in layers.** Only the bottom layer changes, so everything you learn once works everywhere.

Try it

```
make doctor                                  # check your tools
make start                                   # run the demo API locally (terminal 1)
make ask Q="what is GitOps?"                 # ask it (terminal 2)
ls flux/clusters                             # one folder per cluster in the fleet
cat flux/infrastructure/policy/*.yaml        # the rules every cluster gets
```

Words to know

- **GitOps:** operating infrastructure by pull requests, where clusters pull from Git.
- **Drift:** the cluster no longer matching what Git says.
- **Kyverno / OPA Gatekeeper:** policy engines that check every change against your rules.
- **SBOM:** software bill of materials, a list of everything inside an image.
- **OpenCost:** open-source Kubernetes cost monitoring.
- **RTO / RPO:** recovery time objective (max downtime) and recovery point objective (max data loss).

Check yourself

1. In GitOps, how do you roll back a bad change?
2. Why run an OTel agent on every node *and* a gateway per cluster?
3. Why do the policies start in Audit mode?
4. Which recovery pattern should most teams start with?

Answers: 1) `git revert` the change, and the clusters reconcile back. 2) Apps send telemetry locally to the agent, and the gateway adds the cluster name and forwards everything to one place. 3) To see violations without breaking running workloads, then enforce once clean. 4) Active-passive.

Series complete. Your challenge: build it. Start with Part 2 on your laptop this week, then pick one cloud. Put your work on GitHub and share it.

Emmanuel Naweji · emmanuelnaweji.com · transformed2succeed.com
Code: github.com/Here2ServeU/building-ai-infra · Series guide: docs/series/README.md